



AMRITA SCHOOL OF ENGINEERING, BENGALURU

DEPARTMENT OF ELECTRONICS & COMMUNICATION
ENGINEERING

B. TECH. IV SEMESTER AY 2024-2025

23ECE284 MICROCONTROLLER AND INTERFACING

LABORATORY

SYSTEM DESIGN REPORT

JANUARY TO APRIL 2025

ELECTRONIC VOTING MACHING USING LPC2148

Submitted by

<i>REGISTRATION NO.</i>	<i>NAME</i>
BL.EN.U4EAC23020	E R Grishithaa
BL.EN.U4EAC23029	Krishnapriya O
BL.EN.U4EAC23059	Trishala K
BL.EN.U4EAC23076	Vinuthna Reddy Y

BATCH NUMBER: 8

FACULTY EXAMINER SIGNATURE WITH EVALUATION DATE

COMMENTS (IF ANY)

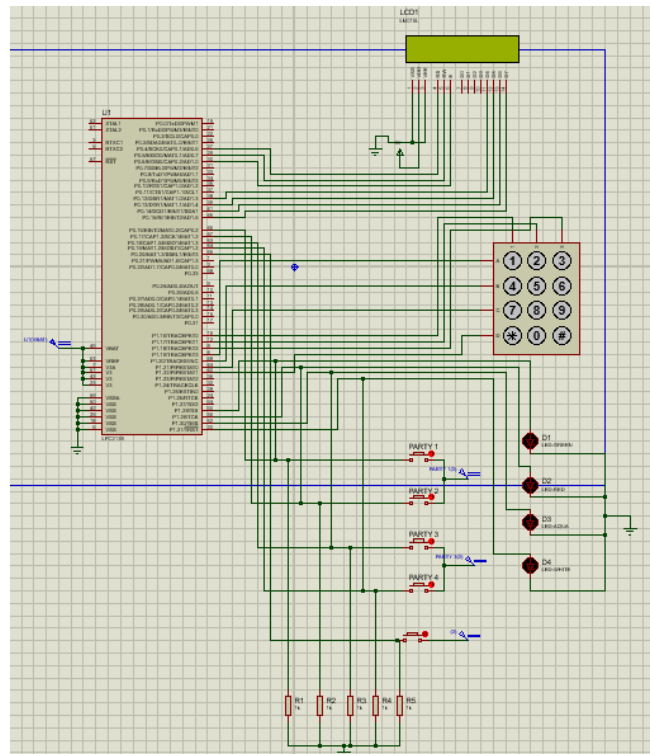
Aim:

To design and implement a secure voting system using LPC2148 ARM7 microcontroller with password-protected access, candidate voting using push buttons, and result display via an LCD module.

Components required along with their specifications:

Component	Specification / Part Number
Microcontroller	LPC2148 (ARM7TDMI-S Core)
LCD Module	16x2 Alphanumeric (LM016L or similar)
Keypad	4x3 Matrix Keypad
Push Buttons	Tactile switches (4 + 1 for end)
LEDs	5mm Red LEDs (4)
Resistors	220Ω, 10kΩ
Simulation Software	Proteus Design Suite 8.x or later

Circuit diagram:



Theory and working principle:

The electronic voting system presented here is designed using the LPC2148 ARM7-based microcontroller. It provides a secure method for voting where each voter is authenticated using a password. This system ensures one person—one vote through authentication, and the vote is cast using dedicated push buttons corresponding to political parties or NOTA (None of the Above). The votes are stored in variables and displayed after the voting process ends, either automatically or manually.

- Microcontroller used: LPC2148 (ARM7TDMI-S)
- Display: 16x2 LCD for instructions and results
- Input devices: 4x3 Keypad for password entry, Push buttons for voting
- Output indicators: LEDs to confirm vote
- End button: Used to terminate voting early

◆ System Initialization

The system starts with initializing peripherals:

- The LCD is set to 4-bit mode and displays a welcome message, followed by a prompt that voting has started.
- The keypad is initialized where rows are configured as outputs and columns as inputs.
- The LEDs and buttons are configured through GPIO directions.
- A fixed array of 10 predefined passwords is used to validate each voter.

Key points:

- LCD connected on P0.12–P0.15 (data), P0.4–P0.6 (control)
- Keypad connected on P1.19–P1.22 (rows), P1.16–P1.18 (columns)
- Voting buttons on P0.16–P0.19; End button on P0.20
- LEDs on P1.28–P1.31 to show vote confirmation

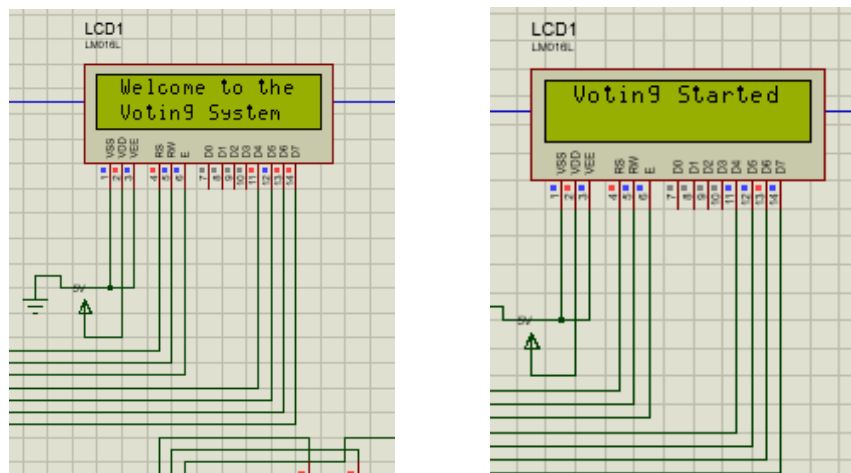


Figure 1: Initializing LCD and getting the Voting system started

◆ Password Authentication

For each voter, the system prompts them to enter their password using the keypad. The user types a 4-digit password followed by the '#' key to confirm. The system compares this input with the stored password.

- If the password is correct, "Access Granted" is shown, and the user proceeds to vote.
- If incorrect, the user gets 3 attempts. After 3 failed attempts, the system shows "Access Denied" and locks the user out.

This mechanism ensures that only authorized individuals can vote and only once.

```
int main() {
```

```
    char passwords[10][5] = {"1234", "2345", "3456", "4567", "5678", "6789", "7890",  
    "8901", "9012", "0123"};
```

Key points:

- Passwords are stored as strings in a 2D array
- Input masked as '*' on the LCD
- Each user has only 3 tries
- After 10 users, voting ends automatically

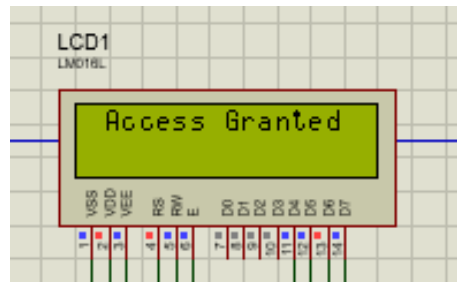
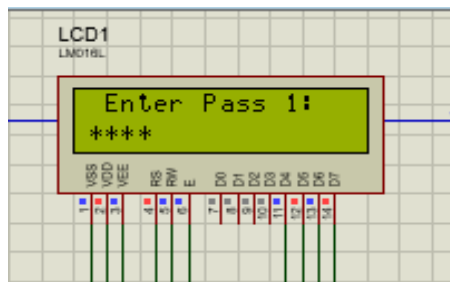


Figure 2a: When password entered is 1234 and press '#' as Enter, the password is authenticated and access is granted.

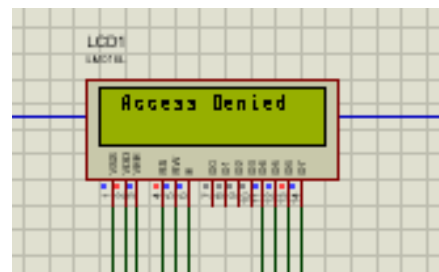
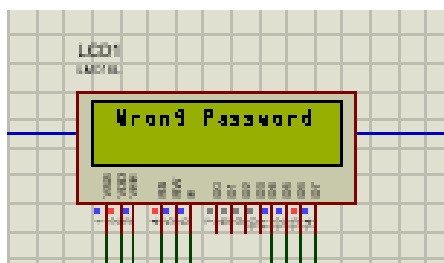


Figure 2b: When password entered is wrong, 3 chances are give to correct and Try again, then access is denied.

❖ Voting Process

Once a voter is authenticated, they are prompted to press a button to vote.

There are four voting options:

- Party 1 (Button at P0.16)
- Party 2 (P0.17)
- Party 3 (P0.18)
- NOTA (P0.19)

When a button is pressed:

- The corresponding vote counter is incremented
- An LED blinks briefly to confirm the vote
- LCD displays “Vote Casted”

This ensures clarity and transparency in the voting process. Each voter can vote only once.

Key points:

- Only one vote allowed per user
- Real-time confirmation via LED and LCD
- Debouncing handled through delays

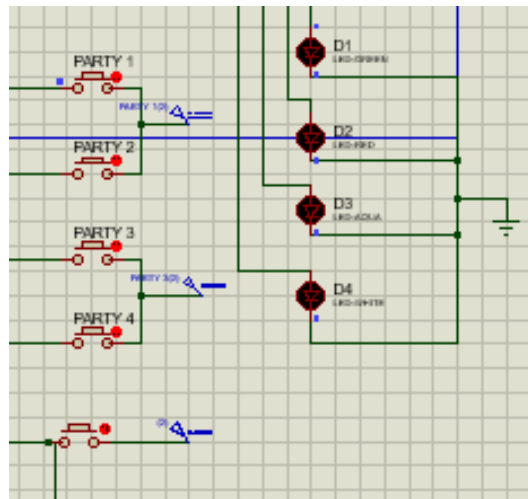


Figure 3a: Depiction of 4 push buttons representing three parties and NOTA. There exists one more button for ending the voting process.

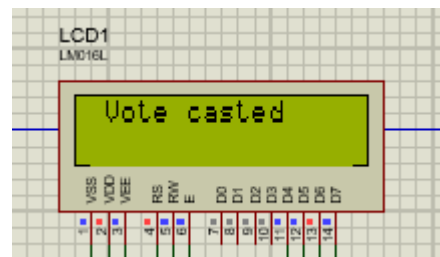
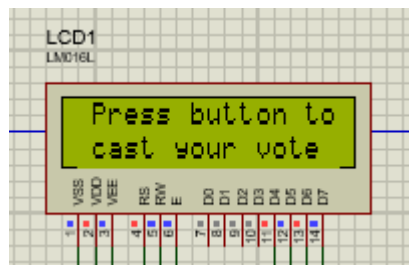


Figure 3b: Depiction of instruction and confirmation displayed to the voter for better convenience

◆ Voting Termination

Voting can end in two ways:

1. **Automatically** after 10 voters have voted.
2. **Manually** by pressing the **END button** (P0.20).

Upon termination, the system moves to result display mode.

◆ Result Display

Once voting ends:

- The LCD shows the vote count for each party and NOTA sequentially.
- A comparison is made to determine the **winner**.
 - If one party has the highest votes, it's declared the winner.
 - If NOTA has the highest votes, “NOTA Majority” is shown.
 - If there's a tie or no clear winner, the system displays “No Clear Winner”.

This ensures an accurate summary of the election outcome.

Key points:

- Results calculated using conditional comparisons
- Votes shown one-by-one with delays
- Final message: “Thank You!”

Program:

```
#include <lpc214x.h>
#include <stdint.h>
#include <string.h>
#include <stdio.h>
// LCD pins
#define LCD_RS 0x00000010 // P0.4
#define LCD_RW 0x00000020 // P0.5
#define LCD_EN 0x00000040 // P0.6
#define LCD_DATA_MASK 0x0000F000 // P0.12 - P0.15

// Keypad pins
#define ROW_MASK ((1<<19) | (1<<20) | (1<<21) | (1<<22)) // P1.19–22
#define COL_MASK ((1<<16) | (1<<17) | (1<<18)) // P1.16–18

// Voting button (End button at P0.20)
#define END_BUTTON_MASK (1 << 20)
```

```

void delay(unsigned int count) {
    unsigned int j, k;
    for (j = 0; j <= count; j++)
        for (k = 0; k <= 600; k++);
}

// LCD functions
void LCD_CMD(char cmd) {
    IO0PIN = (IO0PIN & ~LCD_DATA_MASK) | ((cmd & 0xF0) << 8);
    IO0CLR = LCD_RS | LCD_RW;
    IO0SET = LCD_EN;
    delay(2);
    IO0CLR = LCD_EN;

    IO0PIN = (IO0PIN & ~LCD_DATA_MASK) | ((cmd & 0x0F) << 12);
    IO0CLR = LCD_RS | LCD_RW;
    IO0SET = LCD_EN;
    delay(2);
    IO0CLR = LCD_EN;
}

void LCD_INIT() {
    IO0DIR |= LCD_RS | LCD_RW | LCD_EN | LCD_DATA_MASK;
    delay(50);
    LCD_CMD(0x02);
    LCD_CMD(0x28);
    LCD_CMD(0x0C);
    LCD_CMD(0x06);
    LCD_CMD(0x01);
    delay(2);
}

void LCD_CHAR(char data) {
    IO0PIN = (IO0PIN & ~LCD_DATA_MASK) | ((data & 0xF0) << 8);
    IO0SET = LCD_RS;
    IO0CLR = LCD_RW;
    IO0SET = LCD_EN;
    delay(2);
    IO0CLR = LCD_EN;

    IO0PIN = (IO0PIN & ~LCD_DATA_MASK) | ((data & 0x0F) << 12);
    IO0SET = LCD_RS;
    IO0CLR = LCD_RW;
    IO0SET = LCD_EN;
    delay(2);
}

```

```

    IO0CLR = LCD_EN;
}

void LCD_STRING(char *msg) {
    while (*msg) {
        LCD_CHAR(*msg++);
    }
}

// Keypad
char keypad[4][3] = {
    {'1','2','3'},
    {'4','5','6'},
    {'7','8','9'},
    {'*','0','#'}
};

void keypad_init() {
    PINSEL2 = 0x00000000;
    IO1DIR |= ROW_MASK;
    IO1DIR &= ~COL_MASK;
    IO1SET = ROW_MASK;
}

char get_key() {
    int row, col;
    for (row = 0; row < 4; row++) {
        IO1SET = ROW_MASK;
        IO1CLR = (1 << (19 + row));
        delay(2);
        for (col = 0; col < 3; col++) {
            if (!(IO1PIN & (1 << (16 + col)))) {
                delay(20);
                while (!(IO1PIN & (1 << (16 + col))));
                return keypad[row][col];
            }
        }
    }
    return 0;
}

int main() {
    char passwords[10][5] = {
        "1234", "2345", "3456", "4567", "5678",
        "6789", "7890", "8901", "9012", "0123"
    };
};

```

```

char input[5];
int i, attempts, candidate_count = 0, candidate_id = 0;
char key;
int vote_p1 = 0, vote_p2 = 0, vote_p3 = 0, vote_nota = 0;
char line1[32];

IO0DIR |= (1 << 4) | (1 << 5) | (1 << 6) | LCD_DATA_MASK;
IO0DIR &= ~(1 << 16) | (1 << 17) | (1 << 18) | (1 << 19)); // Voting buttons
IO1DIR |= (1 << 28) | (1 << 29) | (1 << 30) | (1 << 31); // LEDs

LCD_INIT();
keypad_init();

LCD_CMD(0x01);
LCD_CMD(0x80);
LCD_STRING(" Welcome to the");
LCD_CMD(0xC0);
LCD_STRING(" Voting System");
delay(10000);

LCD_CMD(0x01);
LCD_CMD(0x80);
LCD_STRING(" Voting Started");
delay(10000);

while (candidate_count < 10) {
    int access_granted = 0;
    candidate_id++;
    LCD_CMD(0x01);
    LCD_CMD(0x80);
    sprintf(line1, " Candidate %d", candidate_id);
    LCD_STRING(line1);
    delay(15000);

    LCD_CMD(0x01);
    LCD_CMD(0x80);
    sprintf(line1, " Enter Pass %d:", candidate_id);
    LCD_STRING(line1);
    LCD_CMD(0xC0);

    attempts = 0;
    while (attempts < 3) {
        i = 0;
        memset(input, 0, sizeof(input));

        while (1) {

```

```

    key = get_key();
    if (key) {
        if (key == '#') {
            input[i] = '\0';
            break;
        } else if (i < 4) {
            input[i++] = key;
            LCD_CHAR('*');
        }
    }
}
}
if (strcmp(input, passwords[candidate_count]) == 0) {
    access_granted = 1;
    LCD_CMD(0x01);
    LCD_CMD(0x80);
    LCD_STRING(" Access Granted");
    delay(15000);
    break;
} else {
    attempts++;
    LCD_CMD(0x01);
    LCD_CMD(0x80);
    LCD_STRING(" Wrong Password");
    delay(15000);

    if (attempts >= 3) {
        LCD_CMD(0x01);
        LCD_CMD(0x80);
        LCD_STRING(" Access Denied");
        delay(15000);
        LCD_CMD(0x01);
        LCD_STRING(" Locked");
        delay(15000);
    } else {
        LCD_CMD(0x01);
        LCD_CMD(0x80);
        LCD_STRING(" Try Again:");
        LCD_CMD(0xC0);
    }
}
}
if (!access_granted) {
    candidate_count++;
    continue;
}

```

```

LCD_CMD(0x01);
LCD_CMD(0x80);
LCD_STRING(" Press button to");
LCD_CMD(0xC0);
LCD_STRING(" cast your vote");
delay(15000);

while (1) {
  if (IO0PIN & (1 << 16)) { // Party 1
    vote_p1++;
    IO1SET = (1 << 28);
    LCD_CMD(0x01);
    LCD_STRING(" Vote casted");
    delay(15000);
    IO1CLR = (1 << 28);
    break;
  } else if (IO0PIN & (1 << 17)) { // Party 2
    vote_p2++;
    IO1SET = (1 << 29);
    LCD_CMD(0x01);
    LCD_STRING(" Vote casted");
    delay(15000);
    IO1CLR = (1 << 29);
    break;
  } else if (IO0PIN & (1 << 18)) { // Party 3
    vote_p3++;
    IO1SET = (1 << 30);
    LCD_CMD(0x01);
    LCD_STRING(" Vote casted");
    delay(15000);
    IO1CLR = (1 << 30);
    break;
  } else if (IO0PIN & (1 << 19)) { // NOTA
    vote_nota++;
    IO1SET = (1 << 31);
    LCD_CMD(0x01);
    LCD_STRING(" Vote casted");
    delay(15000);
    IO1CLR = (1 << 31);
    break;
  } else if (IO0PIN & END_BUTTON_MASK) {
    LCD_CMD(0x01);
    LCD_CMD(0x80);
    LCD_STRING(" Voting Ended");
    delay(15000);
    goto show_results;
  }
}

```

```

    }
  }
  candidate_count++;
}
show_results:
  LCD_CMD(0x01);
  LCD_CMD(0x80);
  LCD_STRING(" Voting Time Over");
  LCD_CMD(0xC0);
  LCD_STRING(" Showing Results");
  delay(10000);

  LCD_CMD(0x01);
  LCD_CMD(0x80);
  sprintf(line1, " Party1:%d ", vote_p1);
  LCD_STRING(line1);
  delay(10000);

  LCD_CMD(0x01);
  LCD_CMD(0x80);
  sprintf(line1, " Party2:%d ", vote_p2);
  LCD_STRING(line1);
  delay(10000);

  LCD_CMD(0x01);
  LCD_CMD(0x80);
  sprintf(line1, " Party3:%d ", vote_p3);
  LCD_STRING(line1);
  delay(10000);

  LCD_CMD(0x01);
  LCD_CMD(0x80);
  sprintf(line1, " NOTA:%d ", vote_nota);
  LCD_STRING(line1);
  delay(10000);

  LCD_CMD(0x01);
  LCD_CMD(0x80);
  if (vote_p1 > vote_p2 && vote_p1 > vote_p3 && vote_p1 > vote_nota)
    LCD_STRING(" Winner: Party 1");
  else if (vote_p2 > vote_p1 && vote_p2 > vote_p3 && vote_p2 > vote_nota)
    LCD_STRING(" Winner: Party 2");
  else if (vote_p3 > vote_p1 && vote_p3 > vote_p2 && vote_p3 > vote_nota)
    LCD_STRING(" Winner: Party 3");
  else if (vote_nota > vote_p1 && vote_nota > vote_p2 && vote_nota > vote_p3)
    LCD_STRING(" NOTA Majority");

```

```

else
    LCD_STRING(" No Clear Winner");

delay(10000);
LCD_CMD(0x01);
LCD_STRING(" Thank You!");
while (1);
}

```

Simulator used for simulator-based output: Proteus 8 Professional

Screenshot of the simulator-based circuit implementation:

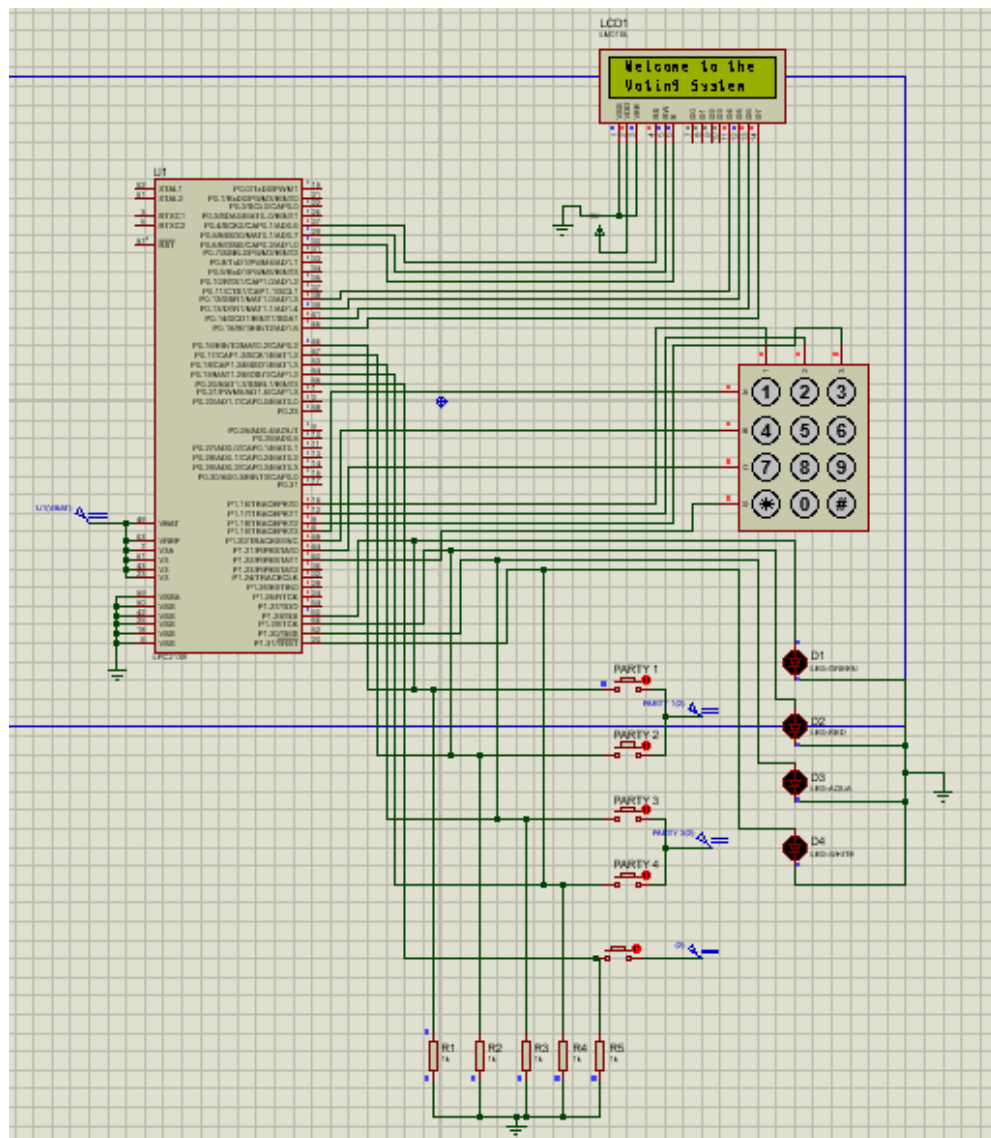


Figure 4: Running simulation of the Electronic Voting System Circuit

Inputs applied to the simulator:

- **4x3 Matrix Keypad Inputs:**

Simulated keypresses on the keypad to enter the 4-digit password for each voter.
E.g., pressing keys 1, 2, 3, 4, followed by # to submit the password.

- **Voting Buttons:**

Tactile push buttons connected to LPC2148 GPIO pins:

- P0.16 for Party 1
- P0.17 for Party 2
- P0.18 for Party 3
- P0.19 for NOTA
- P0.20 for ending the voting process

- **LEDs and LCD:**

Controlled through microcontroller outputs to simulate confirmation and display actions respectively.

Output obtained from the simulator:

- **LCD Display:**

- Displayed welcome and voting instructions
- Prompted for password input
- Showed vote confirmation ("Vote Casted")
- Displayed results after voting ended (vote counts and winner)

- **LED Indications:**

- Corresponding LED turned ON briefly when a vote was cast
- Helped visually confirm that a vote was registered correctly

- **Voting Count:**

- Internal counters updated based on button presses
- Final results accurately shown based on simulated inputs

- **Security:**

- Incorrect password attempts denied access after 3 tries
- Only one vote allowed per valid password entry

Conclusion:

This project showcases a **simple, secure, and interactive voting system** using embedded technology. It integrates password protection, button-based voting, and visual feedback

through LEDs and LCDs. The use of LPC2148 enables efficient handling of I/O operations. Such a system can be deployed in small-scale elections, educational institutions, or mock polls to demonstrate the benefits of digital voting.

Highlights:

- One person—one vote ensured with password protection
- Easy to use with visual instructions
- Accurate and tamper-proof result calculation
- Suitable for real-time demonstration and academic use

Problems faced during the implementation:

- **1. Keypad Input Not Detected Properly:**

Initially, the keypad did not return keypresses correctly due to incorrect row/column GPIO configuration and insufficient delays for debouncing.

- **2. LCD Display Flickering or Not Displaying Correctly:**

The LCD would occasionally flicker or show garbage values due to improper initialization in 4-bit mode.

- **3. Vote Count Showing Multiple Times:**

The vote confirmation message was repeating or votes were being registered multiple times due to button bouncing.

- **4. Voting Not Stopping on End Button Press:**

The logic to detect the stop voting button (P0.20) was not working correctly due to missing pull-down resistor configuration or incorrect condition checks

Problems resolved using solutions:

- **1. Keypad Fixes:**

GPIO directions were carefully set (rows as output, columns as input), and adequate delays were introduced after each key press to ensure reliable reading and debouncing.

- **2. LCD Fixes:**

The LCD was reinitialized using a well-tested 4-bit command sequence. Timings were fine-tuned for command/data latching.

- **3. Vote Debounce Solution:**

Software debouncing using delay loops after detecting button press ensured only one vote per press was recorded.

- **4. Stop Button Functionality:**

The stop button pin was added with proper masking and checked in the vote loop to break early and proceed to result display.

Any other comment:

This project provided practical exposure to real-world embedded system applications involving secure input handling, user feedback mechanisms, and decision-making based on input. The system can be further improved by integrating external memory to log each vote, adding a real-time clock to timestamp votes, and implementing wireless communication for centralized result aggregation.

References:

- **NXP Semiconductors.** (2006). *LPC2141/42/44/46/48 User Manual*. [Online PDF]. Available at: <https://www.nxp.com/docs/en/user-guide/UM10139.pdf>
– Comprehensive documentation on LPC2148 microcontroller features and peripheral usage.
- **Labcenter Electronics Ltd.** *Proteus Design Suite – Simulation Software*. [Official Website]. Available at: <https://www.labcenter.com>
– Source for the software used to simulate and validate the embedded circuit.
- **Dogra, R.** (2021). *ARM7 Based Embedded Systems Design*. BPB Publications.
– Practical insights and examples on LPC2148 projects, including interfacing techniques and C code.
- **Hitachi.** (1998). *HD44780U LCD Controller Datasheet*. [Online PDF]. Available at: <https://www.sparkfun.com/datasheets/LCD/HD44780.pdf>
– Detailed hardware specs for the commonly used 16x2 LCD module.
- **Puneeth, S. R., & Ravishankar, M. C.** (2020). “Design and Implementation of a Secure Electronic Voting System using ARM7.” *International Journal of Engineering Research & Technology (IJERT)*, Vol. 9, Issue 5.
– A peer-reviewed paper related to secure voting using LPC2148.
- **YouTube: Edgex Technologies.** “Password Based Electronic Voting Machine using ARM7.” [Online Video]. Available at: <https://www.youtube.com/watch?v=z8gtrGxP1QU>
– Practical demonstration and explanation of a similar voting project.
